Pystachy speed vs surface

Performance against the amount of unmodified Python that Pystachy runs, across 34 commits of main from the project's first day (8 October 2026) to 11 November 2026; dates are UTC. Measured 12 November 2026; scripts, raw data and this report are in research/speed-vs-surface-2026-11-12 on the branch claude/peaceful-faraday-2yr9q6.

Pystachy now runs **397** of the 5,905 CPython test methods that this sweep extracts unmodified, up from **15** on its first day. On the pyperformance ports that have worked since mid-October, its ahead-of-time compiled code is **11.9×** as fast as CPython 3.13 (geometric mean).

CPython test methods passing	Language and builtins tests passing	pyperformance core set, AOT vs CPython	Day-one microbenchmarks, AOT vs CPython
397	218	11.9×	18.0×
6.7% of 5,905, Nov 11	8.9% of 2,445, Nov 11	16 ports; 7.4× on Oct 15	18.1× on Oct 8, same six programs

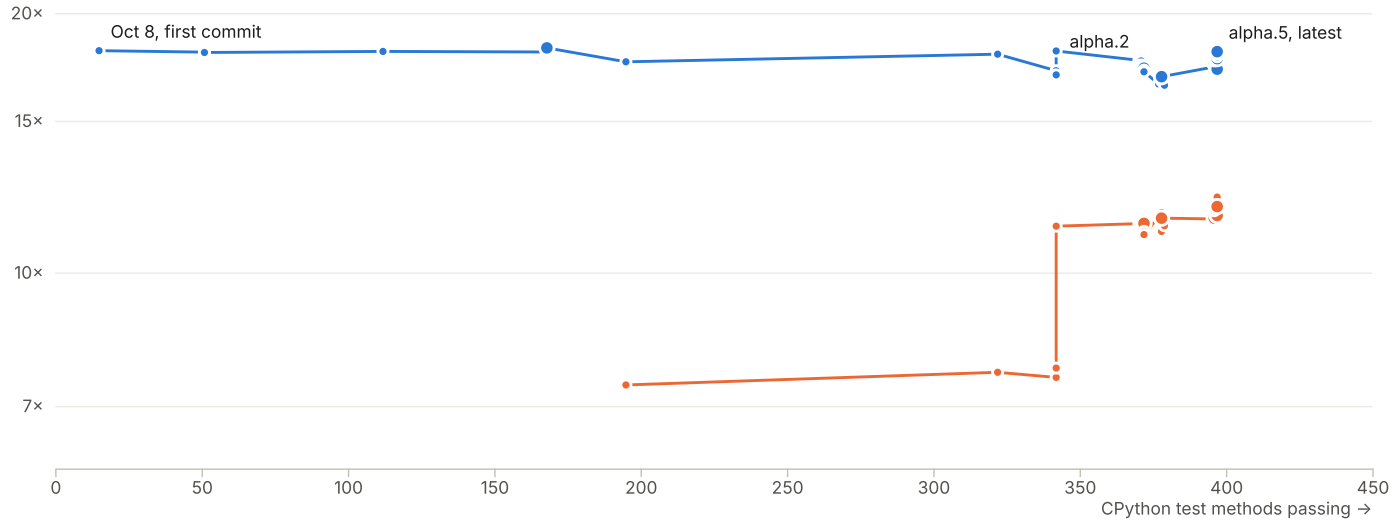
Speed against Python surface, one point per sampled commit

Each point is a commit on main, built and measured on the same machine. Across: how many extracted CPython test methods it runs with CPython's output. Up: the geometric-mean speedup of its AOT executables over CPython 3.13 on the same sources (log scale). Lines join commits in time order.

Surface measure **All CPython test methods passing (of 5,905)**

pyperformance ports, fixed core set — 6 microbenchmarks from day one

× CPython 3.13 speed

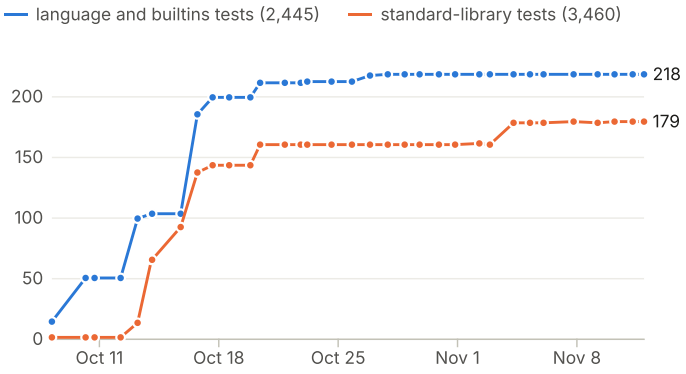


Surface and speed over time

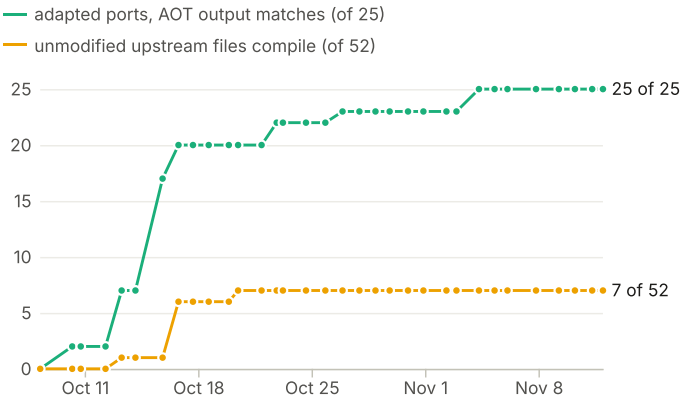
By v0.1.0-alpha.2 on **Oct 20**, **371** methods passed. In the **23** days after that release, **26** more did, while the compiler grew by **20,417** lines.

Fixed workloads: 5,905 CPython 3.13.16 unittest methods (extracted mechanically, qualified on CPython), the 25 adapted pyperformance ports, and 52 unmodified upstream pyperformance files with only pyperf's runner replaced (compile only).

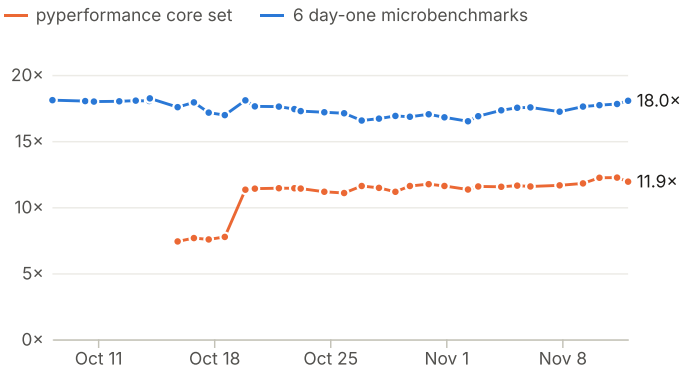
CPython test methods passing



Benchmarks that work



Performance index, AOT vs CPython 3.13

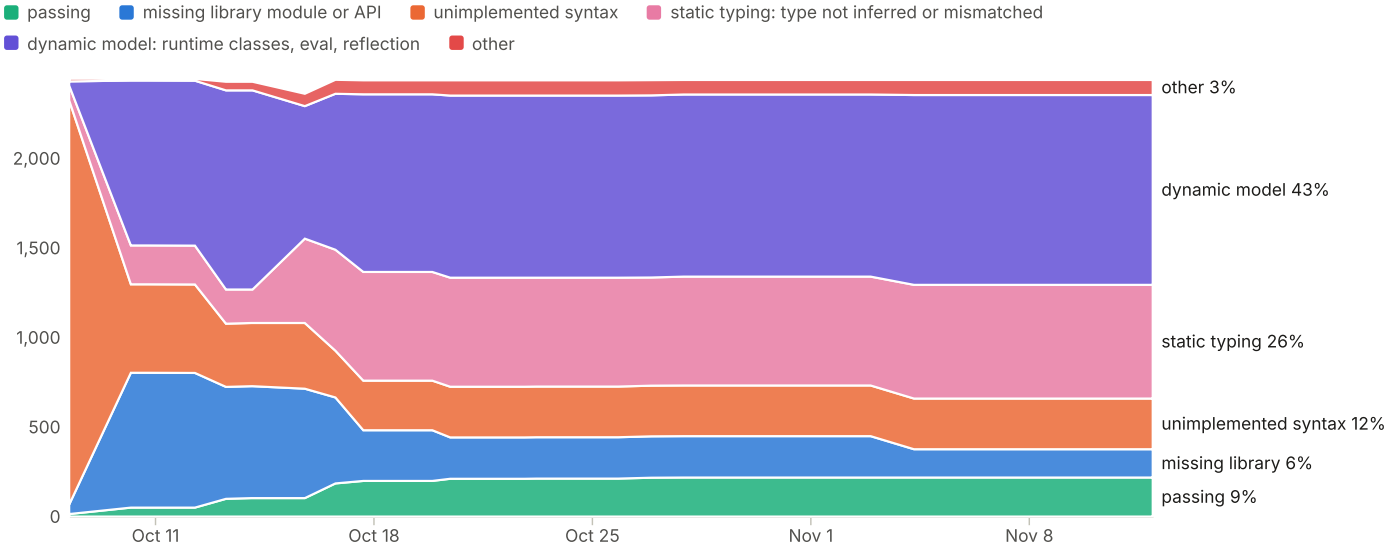


What stops the remaining CPython tests

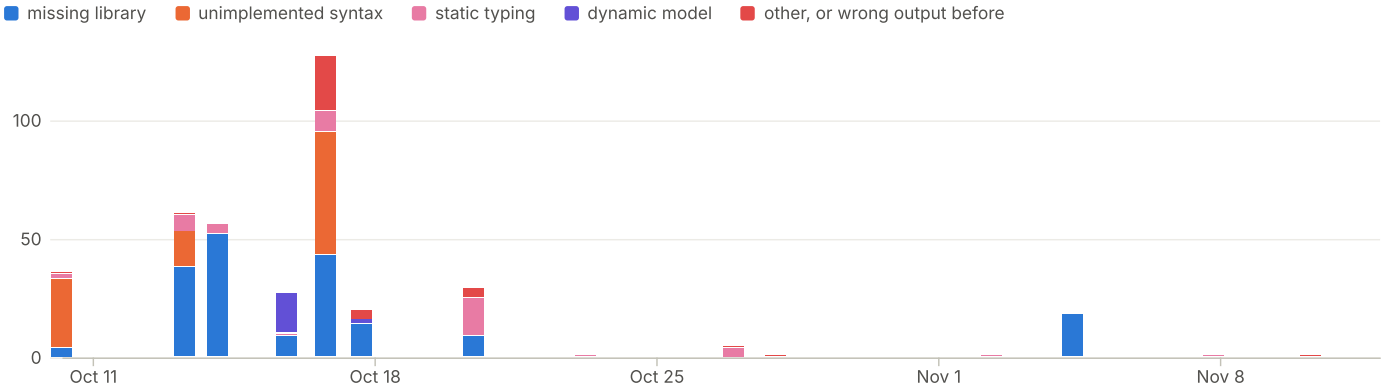
At the latest commit, static-typing and dynamic-model errors are **76%** of the first errors in the language and builtins tests. Since v0.1.0-alpha.2, **0** methods blocked by the dynamic model and **7** blocked by static typing have started to pass.

The first compiler error of every method that does not compile, grouped by cause. Library and syntax gaps are unbuilt work. Typing and dynamic-model errors come from the typed, interpreter-free design: values whose single static type cannot be inferred, and classes, modules or code used as runtime values. Imports come first in a program, so a missing module hides any later error.

Tests **Language and builtins tests (2,445)**



Newly passing methods at each sampled commit, by what blocked them at the previous one



Most common first errors at the latest commit

#	First compiler error	Cause	Methods
1	`cannot import name 'member' from 'enum'`	missing library	724
2	`a class inside a function is not supported`	dynamic model	365
3	`module 'configparser' is not supported: it is not a builtin module`	missing library	288
4	`cannot infer the type of field 'set'; annotate it (self.set: T = ...)`	static typing	253
5	`module 'pickle' is not supported: it is not a builtin module`	dynamic model	238
6	`module 'decimal' is not supported: it is not a builtin module`	missing library	233
7	`'match' statements are not supported`	unimplemented syntax	216
8	`unsupported base class: single inheritance requires a previously declared ordinary class or a supported exception class`	dynamic model	177
9	`cannot import name 'verify' from 'enum'`	missing library	151
10	`module 'gc' is not supported: it is not a builtin module`	dynamic model	141
11	`module '_pyio' is not supported: it is not a builtin module`	missing library	134
12	`module 'locale' is not supported: it is not a builtin module`	missing library	127
13	`module 'pprint' is not supported: it is not a builtin module`	missing library	95
14	`inheritance with generic class templates is not supported: annotate __init__ parameters with concrete types (not Any or object)`	dynamic model	87
15	`cannot import name 'TemporaryFile' from 'tempfile'`	missing library	87

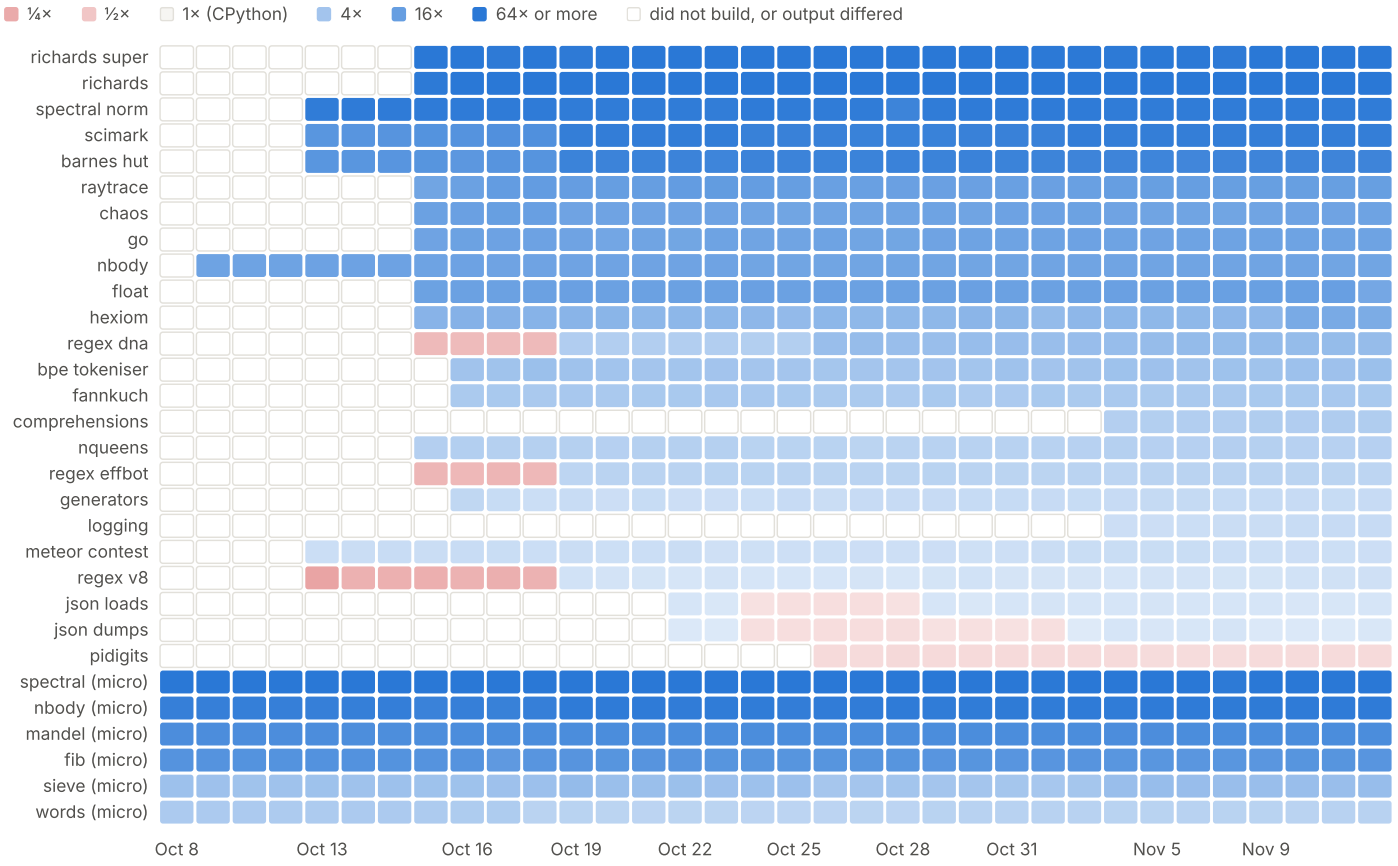
Would CPython's own standard library close the library gap?

At the latest commit, CPython 3.13.16's pure-Python standard library was put on the import path (the repository's `import_sweep.py --overlay`, which leaves out the modules Pystachy ships). Of the **2,429** methods blocked by a missing library piece, **0** pass. Their next errors are below. Opting every test program in to dynamic `Any` instead (`# pystachy: dynamic-any`) leaves **397** passing, because that mode changes only values annotated `Any`.

Where the method now stops	Cause	Methods
now rejected inside CPython's library code	a module or name still missing	708
now rejected inside CPython's library code	dynamic model (generic bases, redefinitions, class-body statements)	219
now rejected inside CPython's library code	static typing (type not inferred)	2
still rejected in the test itself	a module or name still missing	1,323
still rejected in the test itself	dynamic model (generic bases, redefinitions, class-body statements)	141
still rejected in the test itself	static typing (type not inferred)	21
still rejected in the test itself	unimplemented syntax	8
still rejected in the test itself	other	7

Speed by benchmark and commit

AOT speed relative to CPython 3.13 on the same source. Blue is faster than CPython, red is slower, and an empty cell means that commit could not build the benchmark or printed different output. Each row is a fixed source, so color changes along a row are compiler changes.



Largest speed changes between sampled commits, and their causes

Benchmark	From	To	ms before	ms after	Change	Evidence
regex dna	Oct 18	Oct 19	1,506.2	196.0	Faster: lib/re matcher work (#676, #680, #682, #683, #685)	interval of 32 merges, Oct 18 to 19; not bisected
regex effbot	Oct 18	Oct 19	1,032.1	168.3	Faster: same lib/re work	same interval; not bisected
regex v8	Oct 18	Oct 19	1,046.9	235.4	Faster: same lib/re work	same interval; not bisected
scimark	Oct 18	Oct 19	19.2	10.1	Faster: likely field TBAA (#724) or tuple returns by value (#727)	same interval; not bisected
barnes hut	Oct 18	Oct 19	24.9	13.4	Faster: likely field TBAA (#724) or tuple returns by value (#727)	same interval; not bisected
words (micro)	Oct 17	Oct 18	59.3	77.7	Slower: Unicode code-point strings, stage 1 (#667)	bisected over 15 merges to a355c83f
float	Nov 11	Nov 11	32.7	43.5	Slower: listguard bounds-check hoisting with Lean checks (#1268)	bisected over 7 merges to b40ee875

▼ The 52 unmodified upstream pyperformance files: which compile, and what stops the rest

Benchmark	Compiles unmodified	First sampled commit that compiled it	First error at the latest commit
fannkuch	yes	Oct 16	–
generators	yes	Oct 16	–
nbody	yes	Oct 16	–
nqueens	yes	Oct 16	–
regex_v8	yes	Oct 13	–
spectral_norm	yes	Oct 20	–
unpack_sequence	yes	Oct 16	–
2to3	no	—	`module 'subprocess' is not supported: it is not a builtin module`

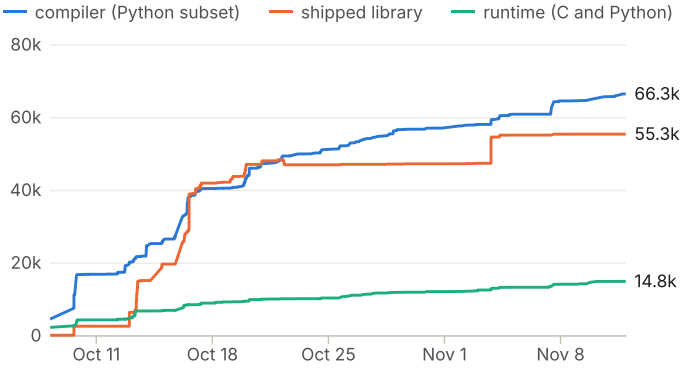
Benchmark	Compiles unmodified	First sampled commit that compiled it	First error at the latest commit
argparse	no	—	`unsupported type annotation`
async_generators	no	—	`async functions are not supported`
async_tree	no	—	`module 'asyncio' is not supported: it is not a builtin module`
asyncio_tcp	no	—	`module 'asyncio' is not supported: it is not a builtin module`
barnes_hut	no	—	`cannot infer the type of field 'particles'; annotate it (self.particles: T = ...)`
base64	no	—	`module 'base64' is not supported: it is not a builtin module`
bpe_tokeniser	no	—	`cannot infer the type of field '_decoder'; annotate it (self._decoder: T = ...)`
chaos	no	—	`cannot infer the type of class attribute '__rmul__'; annotate it (__rmul__: T = ...)`
comprehensions	no	—	`cannot infer the type of 'widgets', an empty list so far: annotate it (widgets: list[T] = [])`
concurrent_imap	no	—	`module 'multiprocessing' is not supported: it is not a builtin module`
coroutines	no	—	`async functions are not supported`
crypto_pyaes	no	—	`module 'pyaes' is not supported: it is not a builtin module`
decimal_factorial	no	—	`module 'decimal' is not supported: it is not a builtin module`
decimal_pi	no	—	`module 'decimal' is not supported: it is not a builtin module`
deepcopy	no	—	`unsupported type annotation`
deltablue	no	—	`unsupported base class: single inheritance requires a previously declared ordinary class or a supported exception class`
float	no	—	`cannot infer the type of field 'x'; annotate it (self.x: T = ...)`
gc_collect	no	—	`module 'gc' is not supported: it is not a builtin module`
gc_traversal	no	—	`module 'gc' is not supported: it is not a builtin module`
go	no	—	`cannot infer the type of field 'pos_child'; annotate it (self.pos_child: T = ...)`
hexiom	no	—	`class attribute 'HIGHEST_VALUE_STRATEGY' of 'Done' used in a class-body default is not supported`
json_dumps	no	—	`cannot infer the type of an empty dict; add a type annotation (d: dict[K, V] = {}, or -> dict[K, V] on a function returning the dict)`
json_loads	no	—	`expected int, got str`
logging	no	—	`expected <function bench_silent>, got <function bench_simple_output> (each function is a type of its own: a variable, a list's items or a`
mdp	no	—	`module 'fractions' is not supported: it is not a builtin module`
meteor_contest	no	—	`expected int, got float`
pathlib	no	—	`'pathlib.PosixPath' object has no method 'stat'`
pickle	no	—	`module 'pickle' is not supported: it is not a builtin module`
pidigits	no	—	`map() is supported as the argument of list(), tuple() (with scalar or optional scalar items), bytes(), sorted(), sum(), min(), max(), any`
pprint	no	—	`module 'pprint' is not supported: it is not a builtin module`
pyflate	no	—	`module 'hashlib' is not supported: it is not a builtin module`
python_startup	no	—	`module 'subprocess' is not supported: it is not a builtin module`
raytrace	no	—	`module 'array' is not supported: it is not a builtin module`
regex_compile	no	—	`cannot infer the type of 'regexes', which capture_compile() reads, where it is defined: annotate it (regexes: T = ...)`
regex_dna	no	—	`unsupported call bytearray()`

Benchmark	Compiles unmodified	First sampled commit that compiled it	First error at the latest commit
regex_effbot	no	—	`type <function bench_regex_effbot> has no attribute 'data'`
richards	no	—	`inheritance with generic class templates is not supported: annotate __init__ parameters with concrete types (not Any or object)`
richards_super	no	—	`inheritance with generic class templates is not supported: annotate __init__ parameters with concrete types (not Any or object)`
scimark	no	—	`module 'array' is not supported: it is not a builtin module`
sqlite_synth	no	—	`module 'sqlite3' is not supported: it is not a builtin module`
telco	no	—	`module 'decimal' is not supported: it is not a builtin module`
tomli_loads	no	—	`module 'tomli' is not supported: it is not a builtin module`
typing_runtime_protocols	no	—	`unsupported base class: single inheritance requires a previously declared ordinary class or a supported exception class`
xml_etree	no	—	`module 'xml' is not supported: it is not a builtin module`

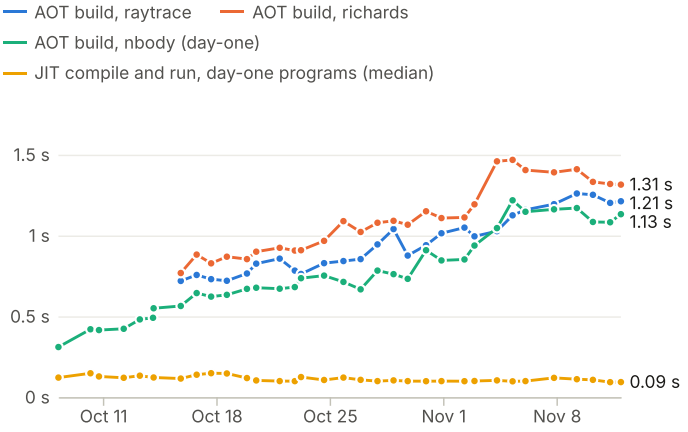
What the surface cost

Size of the compiler, runtime and shipped library at every commit on main, and compile time measured on the idle machine: seconds for `pystachy build` to produce an executable from three fixed programs (median of two, pinned), and for the JIT to compile and run the six day-one programs end to end (median, program cache off).

Lines of source



Compile time for fixed programs



▼ How this was measured

- Commits: end-of-day points on main and every release tag from 7 October to 11 November 2026, each built with its own bootstrap (CPython 3.13 hosting stage 1, LLVM 18.1.3) in a separate worktree and checked at its own fixed point.
- CPython test methods: today's tools/cpython_test_methods.py extracted 9,855 unittest methods from the 144 pinned CPython 3.13.16 test files of tests/upstream/core-methods.sha256; 5,905 complete under CPython 3.13.18 and form the denominator. A method passes when the commit's JIT prints CPython's stdout and stderr and exits the same way, including on a retry with promoting ints. The latest commit reproduces the repository's own census within one method.
- Language and builtins tests are the 60 files that test syntax, builtins and core types (test_set, test_patma, test_builtin, test_exceptions and the like); the rest test standard-library modules.
- Performance: today's 25 adapted pyperformance ports and the six original microbenchmarks, built AOT by each commit and run pinned to one core on an otherwise idle 4-vCPU x86-64 VM, every commit interleaved in each repetition with CPython 3.13.18. Ports report pyperf's timed region; microbenchmarks report process wall time. Every run's output must equal CPython's. unpack_sequence is left out because AOT removes its timed loop.
- Repetitions: five per port and seven per microbenchmark at every commit, medians reported. Between a benchmark's second-fastest and second-slowest run the spread was typically 1–10% (up to about 25% for the shortest programs), so index changes under about 5% are within noise.

- The core set is the 16 ports whose output matched at every sampled commit since the first one with ten working ports (15 October). The pyperformance index uses only that set, so its points compare the same programs.
- Compile time: `pystachy build` of nbody (day-one), richards and raytrace, two runs each pinned to one CPU; JIT end-to-end runs of the six day-one programs with the program cache off and a warm runtime cache.
- Causes are assigned from the text of the first compiler error. A method needs every blocker removed, and only the first is visible, so the categories show what is in front, not everything that remains.

Data sources

Sampled commits	One row per sampled commit on main from 7 October to 11 November 2026: CPython test methods passing, first-error causes, benchmarks that work, source size and compiler size.
AOT speed per benchmark and commit	Median AOT time of each benchmark at each sampled commit and of CPython 3.13.18 on the same source, measured pinned to one core on an idle 4-vCPU VM; speedup is CPython's time over the commit's.
Source size at every commit on main	Lines of compiler, shipped library and runtime source, and differential test counts, at each of the 660 first-parent commits on main.
Most common first errors, latest commit	The 15 most frequent first compiler errors among the CPython test methods v0.4.0-alpha.5 rejects, with the cause each was assigned.
Newly passing methods per sampled commit	Methods that pass at a sampled commit but not at the previous one, by the cause of their first error at the previous one.
Unmodified upstream pyperformance files	Whether each of the 52 pyperformance 1.14.0 benchmark files compiles with only pyperf's runner replaced, the first sampled commit that compiled it, and its first error at the latest commit.
Largest speed changes and their causes	The biggest AOT time changes between consecutive sampled commits, with the merge that caused each where a bisection over the first-parent merges in between found it, or the merges the interval contains.
Latest commit with CPython's standard library added	Where the methods blocked by a missing library piece stop when CPython 3.13.16's pure-Python standard library is on the import path, at v0.4.0-alpha.5.
Performance index per commit (calculated)	Geometric mean of the per-benchmark AOT speedups over CPython 3.13 at each sampled commit: over the 16-port pyperformance core set where all 16 were measured, and over the six day-one microbenchmarks.
Findings since v0.1.0-alpha.2 (calculated)	Methods passing at v0.1.0-alpha.2 and gained since, compiler lines added since, the share of typing and dynamic-model first errors in language tests at the latest commit, and methods unlocked from each since the release.